Baseline vs. Event-Driven Autoscaling in Kubernetes: A Controlled Study of HPA and KEDA for a Queue-Backed Microservice

Group 5
CS5296 Cloud Computing, Spring 2026, City University of Hong Kong

ZHAO Yinhuan SID: 59546734
QIN Haoting SID: 60023963
ZHANG Weiyi SID: 59747335

April 2026

Abstract

Queue-backed microservices are common in cloud systems, but they are difficult to scale with a CPU-only signal. A growing message queue represents unfinished work before the consumer's CPU history fully reflects it, and a CPU-limited pod can remain "busy" even after the incoming burst has already stopped. This project implements a reproducible technical evaluation of two Kubernetes autoscaling strategies for the same RabbitMQ-backed Spring Boot consumer: the standard Horizontal Pod Autoscaler (HPA), which scales from CPU utilisation, and Kubernetes Event-Driven Autoscaling (KEDA), which scales from RabbitMQ queue length. We deploy the system on a single-node K3s cluster running on an AWS EC2 `m5.large` instance, generate a 10-second flash-crowd workload targeting 1,000 messages/s, and run three trials per strategy. All application code, resource limits, replica bounds, and HPA `behavior` parameters are kept identical; only the scaling signal differs. In our measurements, KEDA reacts 10.4 s faster on average than the CPU-based HPA (63.4 s vs. 73.8 s), reaches peak capacity about 10 s earlier, drains the queue 12.2 s sooner, and begins scaling down during the observation window while HPA remains at six pods. The results show that, for event-driven queue consumers, using backlog as the autoscaling signal provides clearer and earlier information than CPU tuning alone.

I Introduction

Elasticity is a core promise of cloud computing [1]: applications should add capacity when demand rises and release it when demand falls. In many modern systems, demand does not arrive only as direct HTTP requests. It is often buffered through message queues, such as order-processing jobs during an e-commerce flash sale, telemetry events from Internet-of-Things devices, or asynchronous tasks in a machine-learning inference pipeline. In these workloads, the queue depth is an immediate measure of unfinished work. If the consumer deployment reacts late, users may experience long processing delays even though the cluster eventually scales out.

Kubernetes [2] provides the Horizontal Pod Autoscaler (HPA), which adjusts a Deployment's replica count so that a selected resource metric, usually CPU or memory utilisation, approaches a target value [3]. HPA is simple and widely available, but CPU is an indirect and sometimes delayed signal for a queue-backed service. A consumer can be CPU-saturated while the backlog is either growing rapidly or almost drained; CPU alone does not tell the autoscaler how much work remains in the queue.

Kubernetes Event-Driven Autoscaling (KEDA) [4] offers a different approach. Instead of asking only how busy the pods are, KEDA can ask how much work is waiting outside the pods. For RabbitMQ, KEDA can scale a consumer directly from queue length. Internally, KEDA still creates an HPA object, but the metric fed into the HPA is an external business metric rather than a CPU metric. This makes KEDA a useful comparison point for event-driven cloud workloads.

Project nature and research question. This is a technical cloud-computing project: we implement and evaluate existing autoscaling mechanisms rather than proposing a new autoscaling algorithm. The central question is: *for the same RabbitMQ-backed consumer and the same burst workload, how do CPU-based HPA and queue-length-based KEDA differ in scale-up responsiveness, queue-drain performance, and scale-down behaviour?*

Methodology. We built a complete reproducibility artifact on GitHub [5]. The artifact deploys RabbitMQ, a Spring Boot consumer, HPA/KEDA policies, a Python load generator, and metric-collection scripts on a single-node K3s cluster. To make the comparison fair, both strategies use the same Docker image, CPU and memory requests/limits, minimum and maximum replica counts, and HPA `behavior` rules. The only independent variable is the signal used for scaling: CPU utilisation for HPA and RabbitMQ queue length for KEDA.

Main findings. KEDA reacts earlier, reaches useful capacity sooner, drains the queue faster, and is the only strategy that scales down during the observed post-burst window. The main reason is not that KEDA uses a more aggressive control loop. It uses a better signal for this workload. Queue length decreases continuously as work is completed, while CPU remains pinned near the pod limit as long as any backlog remains.

The rest of this report is organized as follows. Section II reviews HPA and KEDA. Section III presents the implemented system. Section IV describes the experiment. Section V analyzes the results. Section VI discusses implications and limitations, and Section VII concludes.

II Background and Related Work

II-A Horizontal Pod Autoscaler

The Kubernetes HPA [3] periodically computes the desired number of replicas for a workload. By default, the controller runs every 15 s and obtains resource metrics from `metrics-server`. For a metric-based target, the simplified control rule is

$$n_{\text{desired}} = \left\lceil n_{\text{current}} \cdot \frac{m_{\text{current}}}{m_{\text{target}}} \right\rceil, \tag{1}$$

where the final result is clamped between `minReplicas` and `maxReplicas`. A `behavior` block can further limit how quickly the HPA scales up or down. For example, a scale-down stabilisation window prevents a short metric dip from immediately removing pods, and a percentage policy limits how many pods can be removed in one period.

When the metric is CPU utilisation, Kubernetes computes usage relative to `requests.cpu`. This design is sensible for CPU-bound synchronous services, but it is less informative for

a queue consumer. A pod with `requests.cpu=100 m` and `limits.cpu=300 m` can report approximately 300% utilisation whenever it is continuously busy. The HPA can see that the pod is busy, but it cannot distinguish a queue containing 5,000 messages from a queue containing 200 messages.

II-B Kubernetes Event-Driven Autoscaling

KEDA [4] extends Kubernetes through the `ScaledObject` custom resource. It provides scalers for message brokers, databases, HTTP endpoints, and other event sources [6]. For RabbitMQ, the scaler can read queue length and expose it to Kubernetes as an external metric. KEDA then creates and manages an HPA behind the scenes, so the Kubernetes scaling mechanism remains familiar while the metric becomes workload-aware.

For a RabbitMQ `QueueLength` trigger, the operator specifies a target number of queued messages per replica, denoted as T. If the current queue depth is Q, the intuitive desired replica count is

$$n_{\text{desired}} \approx \left\lceil \frac{Q}{T} \right\rceil . \tag{2}$$

This signal is closer to the service-level problem: the autoscaler responds to the amount of unfinished work rather than to the CPU symptoms of processing that work.

II-C Related Work

Autoscaling has been studied through taxonomies of reactive and predictive cloud scaling algorithms [7], Kubernetes performance models [8], HPA-focused experiments [9], and large-scale serverless workload studies [10]. Our study has a narrower but controlled goal. We do not compare many algorithms. Instead, we hold the application, broker, hardware, resource limits, replica bounds, HPA behaviour settings, and load pattern constant, so that the measured difference can be attributed to the scaling signal itself.

III System Design

III-A Architecture Overview

Fig. 1 summarizes the implemented system. A single AWS EC2 `m5.large` instance (2 vCPU, 8 GiB RAM) hosts a lightweight K3s [11] v1.30 cluster. The cluster contains three application namespaces: `rabbitmq` for the shared message broker, `baseline` for the CPU-based HPA experiment, and `keda` for the queue-length-based KEDA experiment. RabbitMQ exposes two durable queues, `baseline-queue` and `keda-queue`, so that each strategy is tested with an isolated backlog while sharing the same broker installation.

A Python producer running from the evaluator's machine sends AMQP messages to either queue. During each trial, `scripts/collect-metrics.sh` polls Kubernetes and the RabbitMQ Management API [12] at approximately 1 Hz. It records pod counts, pod readiness, queue depth, ingress/egress rates, and cumulative published/delivered messages into CSV files. The same collection script is used for both strategies.

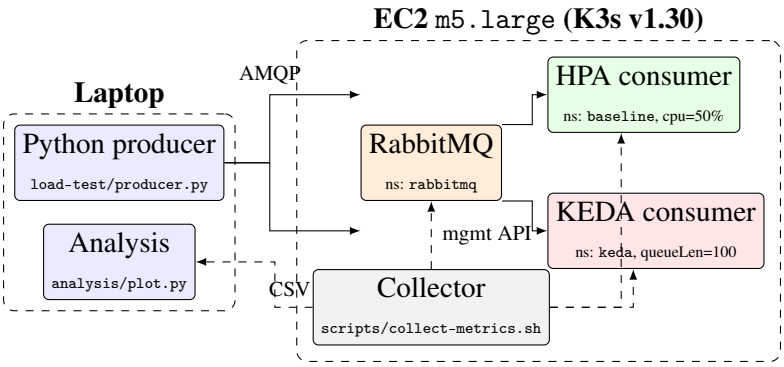

Figure 1. End-to-end architecture. Both experiment groups consume from RabbitMQ using the same Spring Boot image; the only controlled difference is the scaling metric.

III-B Consumer Application

The consumer is a Spring Boot [13] application packaged as the shared image `htasteqin/cs5296-consumer:v1.0`. The main processing logic is in `MessageConsumer.java`. Each message is received through a Spring AMQP `@RabbitListener`, and the handler performs about 200 ms of CPU-bound busy work before acknowledging progress through Micrometer counters and timers.

This artificial CPU-bound work is intentional. If the handler used `Thread.sleep`, the queue could grow while CPU usage stayed low, making the CPU-based HPA fail for a trivial reason. By using busy work, the baseline HPA is given a real CPU signal, while KEDA is given the queue signal. The consumer pods request 100 m CPU and 256 MiB memory and are limited to 300 m CPU and 512 MiB memory. The same values appear in both the `baseline` and `keda` Deployment manifests.

III-C Scaling Strategies

The two strategies share the same replica range and control limits. Both start with one replica and can scale to at most six replicas, a bound chosen to fit the two-vCPU single-node environment without destabilizing RabbitMQ. Both also use the same HPA `behavior` configuration:

- **Scale-up**: no stabilisation window; maximum of `Percent 100%/15s` and `Pods 4/15s`.
- **Scale-down**: 60 s stabilisation window; `Percent 50%/60s` removal policy.

Only the metric differs. The baseline HPA uses CPU utilisation with a 50% target. KEDA uses a RabbitMQ `QueueLength` trigger with target $T = 100$ messages per replica, a 5 s polling interval, and a 60 s cooldown period.

III-D Controlled Variables

The experiment controls every major non-signal variable: the same Docker image, the same Spring Boot code path, the same resource requests and limits, the same per-message processing cost, the same RabbitMQ broker, the same EC2 node, the same burst traffic pattern, and the same metric-collection pipeline. This design prevents a common benchmarking mistake: comparing two autoscalers while also changing the workload, hardware, or application implementation.

IV Experimental Setup

IV-A Environment

The experiment runs on AWS EC2 in `us-west-2`. The node is an `m5.large` instance with 2 vCPU and 8 GiB RAM, running Ubuntu 22.04 LTS. K3s v1.30 provides the Kubernetes cluster, with Helm 3, `metrics-server` v0.7, and KEDA v2.13 installed by the setup scripts. RabbitMQ is deployed from repository-owned Kubernetes manifests using `rabbitmq:3.12-management`. Its readiness and liveness probes use TCP checks on port 5672, which avoids false HTTP-management failures on the constrained single-node cluster.

IV-B Workload

The main load pattern is `load-test/patterns/burst.yaml`. It has three phases: 10 s of warm-up at 5 messages/s, 10 s of burst traffic targeting 1,000 messages/s, and 180 s of silence. The harness script also starts the collector before the producer and keeps observing after the producer exits, so each trial lasts about 390 s and captures roughly 355–360 s after the burst ends.

The producer is implemented in `load-test/producer.py` using the `pika` AMQP client [14]. Each message contains a small JSON payload padded to 256 bytes. In practice, synchronous `basic_publish` reaches around 740 messages/s on the test setup, so the realised burst contains approximately 7,100–7,500 messages rather than exactly 10,000. This ceiling affects both strategies equally because the same producer, pattern file, and broker path are used in all trials.

IV-C Metrics

The collector emits one CSV row per second with eight primary fields: timestamp, total pods, ready pods, queue depth, publish rate, delivery rate, cumulative delivered messages, and cumulative published messages. The analysis script then derives three groups of metrics:

- **Responsiveness**: reaction latency from burst start to the first additional Ready pod, time to peak pod count, and queue-drain time from burst end to `queue_depth=0`.
- **User-visible effect**: average delivery throughput during the drain period and total delivered messages per trial.
- **Resource use**: time to the first scale-down event, final number of Ready pods, peak pod count, and pod-seconds overshoot above the one-replica baseline.

IV-D Trial Protocol

We run three independent trials for each strategy and report the sample mean plus one sample standard deviation. Before every trial, the target Deployment is reset to one replica and the corresponding RabbitMQ queue is purged. The formal sequence is therefore: reset, purge, collect baseline, inject the burst pattern, observe the drain, and record any scale-down event. All six formal trials were executed sequentially on 2026-04-20, and the committed CSV files under `results/raw/` are the data source for the statistics and figures in this report.

Table I. Headline metrics per strategy (mean ± standard deviation, $n = 3$). "—" means the event was not observed within the trial window.

Metric	HPA: CPU signal	KEDA: queue signal
Responsiveness		
Reaction latency (s)	73.8 ± 8.3	$\mathbf{63.4 \pm 3.4}$
Time to peak pods (s)	90.2 ± 4.6	$\mathbf{80.2 \pm 5.1}$
Queue drain time (s)	309.7 ± 5.2	$\mathbf{297.6 \pm 5.6}$
User-visible effect		
Avg. throughput during drain (msg/s)	22.75 ± 0.27	$\mathbf{23.35 \pm 0.27}$
Messages delivered per trial	$7\,451 \pm 57$	$7\,212 \pm 90$
Resource use		
First scale-down (s after burst end)	—	$\mathbf{326.9 \pm 3.1}$
Ready pods at trial end	6.0	**3.0**
Pod-seconds overshoot	$1\,426 \pm 27$	$\mathbf{1\,415 \pm 7}$
Peak pod count	6	6

V Results and Analysis

All results are computed from the six trial CSV files in `results/raw/` using the analysis script `analysis/plot.py`. Unless stated otherwise, each value is the mean over three runs, with one sample standard deviation. Lower values are better for latency, scale-up time, and drain time; lower final pod counts are better after the workload has stopped.

V-A Headline Metrics

Table I shows the aggregate results. KEDA wins on all responsiveness metrics and is the only strategy that releases capacity before the observation window ends. The two strategies reach the same peak of six pods, so the difference is not maximum capacity. The difference is how early each strategy receives useful information about the burst and about the remaining backlog.

The total delivered-message count should be interpreted as descriptive rather than as a direct performance score, because the actual number of messages published varies slightly with producer-side timing. Queue-drain time and delivery throughput are more comparable across runs because they normalize behaviour over the observed drain period.

V-B Responsiveness

Fig. 2 visualizes the three latency-style metrics. Compared with the CPU-based HPA, KEDA reacts 10.4 s earlier on average, which is a 14% reduction in reaction latency. It also reaches the maximum useful replica count about 10 s earlier and clears the queue 12.2 s earlier. These differences are operationally meaningful: in a queue-backed service, every second of delayed scale-out allows the backlog to grow further.

The per-run timeline figures in `results/figures/` show the same pattern in every matched run. KEDA reaches 5–6 Ready pods first, its queue-depth curve stays below the baseline HPA during the drain, and only KEDA removes pods before the trial ends. The aggregate bar chart therefore reflects a consistent run-level pattern rather than a single outlier.

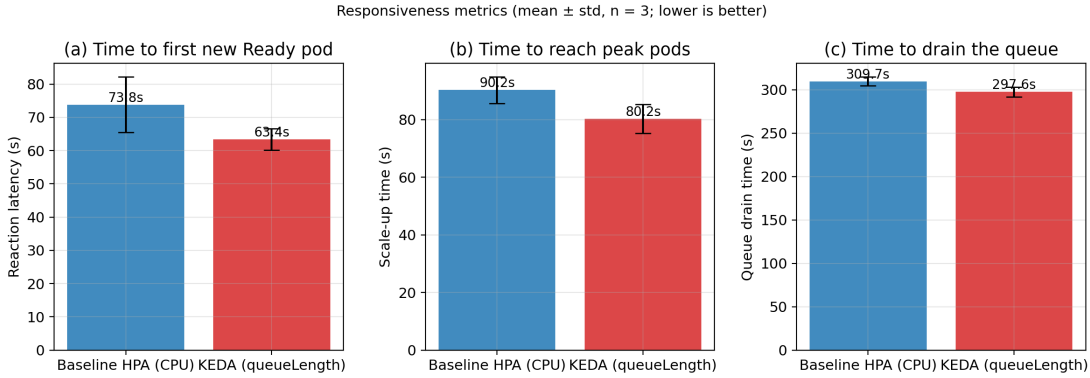

Figure 2. Responsiveness metrics. KEDA responds earlier because queue length is visible as soon as backlog forms, while CPU-based HPA waits for resource metrics to cross the target.

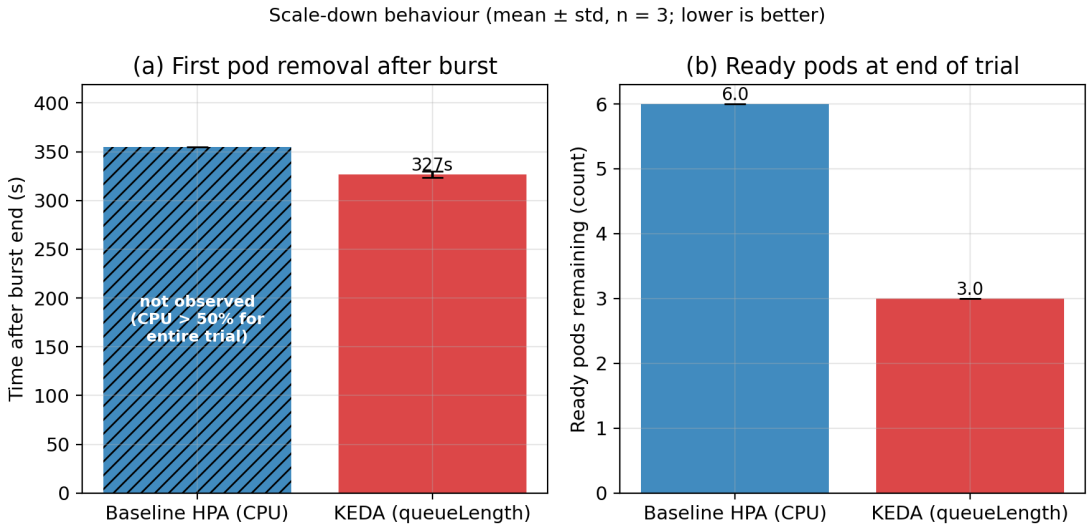

Figure 3. Scale-down behaviour. The CPU-based HPA remains at six pods during the trial, while KEDA begins returning capacity once queue depth falls sufficiently.

V-C Scale-down Behaviour

The strongest qualitative difference appears after the burst stops. Fig. 3 shows that the baseline HPA does not scale down in any trial, while KEDA starts removing pods about 327 s after the burst ends and reaches three Ready pods by the end of the observation window. This is notable because both strategies use the same 60 s scale-down stabilisation window and the same 50%/60 s removal policy.

Why HPA cannot shrink in the observed window. The consumer is CPU-bound, and each pod is limited to 300 m CPU while requesting 100 m. As long as a pod has messages to process, it can remain close to the CPU limit. The HPA therefore observes utilisation near 300% of the CPU request, far above the 50% target. Substituting that ratio into Eq. 1 keeps the desired replica count clamped at `maxReplicas=6` until the queue is almost empty. In other words, the CPU signal tells HPA that the pods are busy, but it does not tell HPA that the backlog is already shrinking.

Why KEDA shrinks earlier. KEDA observes the queue directly. With the target $T = 100$ messages per replica, the desired replica count decreases as Q decreases. Once the queue has dropped far enough, KEDA can recommend fewer than six pods, and the shared 60 s stabilisation window delays but does not prevent the downscale. This is the key technical lesson

of the experiment: using an event signal changes not only scale-up timing but also the ability to detect when extra capacity is no longer needed.

V-D Resource Cost Interpretation

The pod-seconds overshoot is similar for the two strategies: $1\,426 \pm 27$ for HPA and $1\,415 \pm 7$ for KEDA. This does not contradict the scale-down result. Both strategies quickly reach the same six-pod cap, and most of the trial is spent processing the burst, so the cost curves are similar over this short window. KEDA's cost advantage appears in the tail because it starts returning pods earlier. A longer post-drain observation period would likely enlarge the pod-seconds gap.

VI Discussion

The scaling signal is the main design choice. The experiment shows that autoscaling quality is not only a matter of tuning thresholds and cooldown periods. HPA and KEDA share the same Kubernetes HPA control mechanism in this study, and we intentionally keep their scale-up and scale-down policies identical. The observed difference comes from the metric: CPU utilisation is a pod-local symptom, while queue length is a direct measurement of pending work.

Why CPU is a weak signal for this workload. CPU is useful when request pressure and pod CPU usage move together. That assumption breaks down for a queue consumer with a finite backlog. During the drain phase, CPU can stay high even though the workload is becoming smaller every second. This causes the baseline HPA to preserve maximum capacity until the backlog is gone. From a service perspective, that is late information: the controller only learns that the burst is over after the system has already finished handling it.

Practical recommendation. For production queue-backed services, event-driven scaling should be considered the primary scaling path. CPU-based scaling can still be valuable as a guardrail, for example to detect unusually expensive messages, bugs, or unexpected compute pressure. A practical design is therefore hybrid: scale normal elasticity from backlog or lag, and retain resource metrics for safety limits and alerting.

Limitations. This study uses a single-node K3s cluster, one EC2 instance type, one broker, one consumer implementation, and three repetitions per strategy. It is strong enough to demonstrate the directional effect of the signal, but it is not a universal benchmark for all Kubernetes clusters. The post-burst observation window also ends before the baseline HPA scales down, so the reported cost difference is conservative. Future work should test multi-node clusters, longer tails, different queue-length targets, different CPU requests/limits, and HPA driven by external metrics without KEDA.

VII Conclusion

This project implemented a controlled technical evaluation of CPU-based HPA and queue-length-based KEDA for the same RabbitMQ-backed Spring Boot consumer on Kubernetes. With all non-signal parameters held constant, KEDA reacted 10.4 s faster, reached peak capacity earlier, drained the queue 12.2 s sooner, and scaled down during the observation window while HPA stayed at six pods. The reason is that queue length measures unfinished work directly, whereas CPU mainly shows that the current pods are busy. For event-driven cloud workloads, choosing

the right autoscaling signal is therefore more important than only tuning the HPA behaviour parameters.

References

[1] P. Mell and T. Grance, "The NIST definition of cloud computing," National Institute of Standards and Technology, Gaithersburg, MD, USA, Tech. Rep. SP 800-145, 2011.

[2] Cloud Native Computing Foundation, "Kubernetes: production-grade container orchestration," https://kubernetes.io/, 2024, accessed: Apr. 2026.

[3] The Kubernetes Authors, "Horizontal Pod Autoscaling," Kubernetes documentation, https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/, 2024, accessed: Apr. 2026.

[4] KEDA Maintainers, "KEDA: Kubernetes event-driven autoscaling," https://keda.sh/docs/, 2024, cNCF graduated project. Accessed: Apr. 2026.

[5] CS5296 Group 5, "CS5296-Group5-Autoscaling: reproducibility artifact," https://github.com/QinHaoting/CS5296-Group5-Autoscaling, 2026, mIT-licensed open-source artifact.

[6] KEDA Maintainers, "KEDA scalers," https://keda.sh/docs/latest/scalers/, 2024, accessed: Apr. 2026.

[7] C. Qu, R. N. Calheiros, and R. Buyya, "Auto-scaling web applications in clouds: a taxonomy and survey," *ACM Computing Surveys*, vol. 51, no. 4, pp. 1–33, 2018.

[8] V. Medel, O. Rana, J. Á. Bañares, and U. Arronategui, "Modelling performance and resource management in Kubernetes," in *Proc. IEEE/ACM 9th Int. Conf. Utility and Cloud Computing (UCC)*, Shanghai, China, 2016, pp. 257–262.

[9] T.-T. Nguyen, Y.-J. Yeom, T. Kim, D.-H. Park, and S. Kim, "Horizontal pod autoscaling in Kubernetes for elastic container orchestration," *Sensors*, vol. 20, no. 16, p. 4621, 2020.

[10] M. Shahrad, R. Fonseca, Í. Goiri, G. Chaudhry, P. Batum, J. Cooke, E. Laureano, C. Tresness, M. Russinovich, and R. Bianchini, "Serverless in the wild: characterizing and optimizing the serverless workload at a large cloud provider," in *Proc. USENIX Annual Technical Conference (ATC)*, 2020, pp. 205–218.

[11] Rancher Labs, "K3s: lightweight Kubernetes," https://k3s.io/, 2024, accessed: Apr. 2026.

[12] Broadcom Inc., "RabbitMQ: reliable and mature messaging and streaming broker," https://www.rabbitmq.com/, 2024, accessed: Apr. 2026.

[13] VMware, Inc., "Spring Boot reference documentation," https://docs.spring.io/spring-boot/, 2024, accessed: Apr. 2026.

[14] Pika Maintainers, "Pika: Python AMQP 0-9-1 client library," https://pika.readthedocs.io/, 2024, accessed: Apr. 2026.

Artifact Appendix

A.1 Artifact Overview

This appendix describes how to reproduce the project. The artifact contains the Spring Boot consumer, RabbitMQ and Kubernetes manifests, HPA and KEDA scaling policies, Python load-generation scripts, shell scripts for deployment and data collection, raw experimental CSV files, and Python analysis scripts that regenerate the figures and summary table.

A.2 Hardware and Software Dependencies

- **Cloud host**: AWS EC2 `m5.large` with Ubuntu 22.04 LTS. A `t3.medium` can be used for smoke testing, but the reported experiments use `m5.large`.
- **Cluster stack**: K3s v1.30, Helm 3, `metrics-server` v0.7, KEDA v2.13, and RabbitMQ `3.12-management`.
- **Build tools**: Java 17, Maven 3.9, Docker 24.x or compatible Docker Buildx environment.
- **Load-test tools**: Python 3.10+ with `pika`, `click`, and `PyYAML` as listed in the load-test requirements file.
- **Analysis tools**: Python 3.10+ with `pandas` and `matplotlib` as listed in the analysis requirements file.

A.3 Repository

The public repository is:

https://github.com/QinHaoting/CS5296-Group5-Autoscaling

Key directories are:

- `consumer/`: Spring Boot RabbitMQ consumer.
- `k8s/`: namespaces, RabbitMQ, HPA baseline, and KEDA manifests.
- `load-test/`: workload producer and YAML traffic patterns.
- `scripts/`: setup, deployment, collection, run, and teardown scripts.
- `results/`: raw CSV data, generated figures, and `summary.csv`.
- `analysis/`: scripts for metric extraction and plotting.

A.4 Build and Deployment

1. Build and push the consumer image:

```
docker buildx build --platform linux/amd64 \
  -t <dockerhub>/cs5296-consumer:v1.0 --push consumer/
```

2. On the EC2 node, install the cluster dependencies and base services:

```
bash scripts/setup.sh
```

3. Deploy both experiment groups:

```
export CONSUMER_IMAGE=<dockerhub>/cs5296-consumer:v1.0
bash scripts/deploy-baseline.sh
bash scripts/deploy-keda.sh
```

4. Confirm that RabbitMQ, the consumers, HPA, and KEDA are running:

```
kubectl get pods -A
kubectl -n baseline get hpa
kubectl -n keda get scaledobject,hpa
```

A.5 Running the Experiments

Run one trial for each strategy:

```
bash scripts/run-experiment.sh baseline 1
bash scripts/run-experiment.sh keda 1
```

Run the full three-trial batch:

```
for run in 1 2 3; do bash scripts/run-experiment.sh baseline "$run"; done
for run in 1 2 3; do bash scripts/run-experiment.sh keda "$run"; done
```

Each trial resets the Deployment to one replica, purges the target queue, records a short baseline, runs the burst producer, and keeps collecting metrics during the post-burst drain period. The full six-trial batch takes about 49 minutes, excluding manual setup time.

A.6 Reproducing the Figures

After the raw CSV files exist, regenerate the summary and figures with:

```
python analysis/plot.py
```

Expected outputs include `summary.csv`, three per-run timeline figures, and the aggregate figures for responsiveness, scale-down behaviour, and pod-seconds overshoot.

A.7 Expected Results

The regenerated `summary.csv` should show that KEDA reacts approximately 10 s faster than the CPU-based HPA, drains the queue approximately 12 s sooner, and triggers scale-down during the trial. The exact numbers may vary slightly with EC2 performance, RabbitMQ startup time, and producer-side timing, but the qualitative result should remain: queue-length-based scaling reacts earlier and releases capacity earlier for this queue-backed workload.

A.8 Troubleshooting

Additional operational notes are in `docs/RUNBOOK.md`. Common checks include verifying the RabbitMQ NodePort, confirming that the two queues are empty before each trial, ensuring `metrics-server` is reporting CPU metrics, and checking that the KEDA `ScaledObject` has an active RabbitMQ trigger.